

Routing Shim vs KV-Aware Routing

Core Idea

A routing shim is a control layer that sits in front of SGLang (or any LLM serving system). It does more than simply route requests based on KV cache presence — it makes policy decisions about how requests should be handled.

KV-Aware Routing

KV-aware routing answers a single question:

“Which worker already has the useful KV cache?”

It optimizes for:

- Cache hit rate
- Prefix reuse

This is a narrow, single-objective optimization.

Routing Shim

A routing shim evaluates multiple signals:

- priority
- reuse_likelihood
- agent_phase
- expected_output_tokens

Then it makes a broader decision:

“What is the best handling strategy for this request?”

Capabilities of a Shim

A shim can:

- Route requests intelligently
- Enforce priority scheduling
- Balance load across GPUs
- Optimize for latency vs throughput
- Decide when cache reuse matters
- Modify requests (add hints/metadata)
- Log decisions for analysis
- Run experiments on routing strategies

Key Difference

KV-aware routing is a **mechanism**.

The shim is a **policy engine**.

Example Decisions

High reuse request:

- route to same worker
- preserve locality

Low reuse request:

- ignore cache
- route to least busy worker

Architecture

Client → Routing Shim → SGLang → Scheduler → KV Cache → GPU Worker

Conclusion

The real power of a shim is not just improving cache hits — it is enabling intelligent, multi-objective decision-making across the entire serving stack.

